CodeStrike Extended: Can AI Write Secure Web Applications?

Yaash, Ravinder, Deepansh Sharma, Vansh Department of Computer Science COMP 4210 Ethical Hacking

---

Abstract

As developers increasingly rely on Large Language Models (LLMs) to generate production code, a critical question arises: is AI-generated code secure? This research presents *CodeStrike Extended*, a comprehensive security benchmark designed to evaluate vulnerabilities in AI-generated web applications. Expanding upon the BaxBench framework, we generated 4,505 complete microservices across 15 models, 3 frameworks, and 35 OWASP 2025 scenarios. A novel three-layer validation methodology—encompassing the automated CodeStrike framework, OWASP ZAP scanning, and manual penetration testing—was deployed to assess security at scale. Our findings indicate a severe baseline security deficit: without explicit safety prompts, fewer than 1% of functional AI-generated applications were secure. However, introducing specific, vulnerability-aware prompts resulted in up to a 37-fold improvement in security pass rates. Furthermore, comparative analysis revealed that industry-standard tools like OWASP ZAP identified only 14.3% of vulnerabilities compared to human pentesters, while our automated CodeStrike tool achieved 100% precision with 27% recall. The results underscore the necessity of specific safety prompting, rigorous human review, and specialized automated testing when utilizing LLMs for software development.

Keywords

Keywords— Artificial Intelligence, Large Language Models, Web Security, Penetration Testing, OWASP, Static Application Security Testing (SAST), Dynamic Application Security Testing (DAST)

I. INTRODUCTION

The integration of Large Language Models (LLMs) such as GitHub Copilot, OpenAI's ChatGPT, and Anthropic's Claude into software development workflows has revolutionized developer productivity. Current industry surveys suggest that up to 97% of developers utilize AI coding assistants. However, despite their widespread adoption, fewer than 30% of AI-generated code snippets undergo formal security review before being merged into production environments. This creates a significant attack surface, as developers often place unwarranted trust in the security posture of AI outputs.

The primary objective of this project, *CodeStrike Extended*, is to systematically measure the security of AI-generated web applications. We aim to answer three fundamental questions: (1) Which LLM configurations write the most secure code? (2) What is the measurable impact of safety-oriented prompts on code security? (3) How do automated security scanners compare against manual penetration testing for AI-generated code? To address these questions, we constructed a testing pipeline that generates complete containerized web applications, executes functional validations, and subjects the applications to dynamic exploitation across 38 distinct Common Weakness Enumeration (CWE) categories.

This report details the literature informing our methodology, outlines the CodeStrike testing framework, presents the empirical results of our 4,505-app evaluation, and establishes clear recommendations for developers, AI providers, and security teams.

II. LITERATURE REVIEW

The intersection of AI code generation and software security has drawn significant academic and industry scrutiny in recent years. In 2024, Meta introduced *CyberSecEval*, an evaluation suite designed to assess cybersecurity risks in LLMs. CyberSecEval primarily focused on evaluating whether models output insecure code in autocomplete scenarios and their compliance with safety policies when requested to assist in cyberattacks [1]. While valuable, CyberSecEval tests isolated code excerpts rather than functional, end-to-end applications.

To address the need for holistic testing, researchers at ETH Zurich released *BaxBench* (2025), a benchmark for evaluating backend code generation. BaxBench required models to implement complete, self-contained application modules and assessed both functional correctness and vulnerability to real-world exploits [2]. BaxBench found that approximately 5% of AI-generated solutions were both functional and secure. However, as discovered during our research, the original BaxBench framework contained a measurement artifact where applications that immediately crashed were mistakenly categorized as "secure" due to a lack of observable vulnerabilities [3].

Recent empirical studies corroborate the severity of this issue. The 2025 Veracode GenAI Code Security Report found that LLMs introduce security vulnerabilities in up to 45% of test cases, observing that security performance has not improved congruently with functional capability [4]. Similarly, a large-scale analysis of GitHub repositories revealed that vulnerabilities commonly map to the OWASP Top 10, with injection flaws, weak authentication, and missing security controls being the most prevalent [5]. These studies emphasize that iterative prompting can sometimes degrade security if conducted without human oversight [6].

CodeStrike Extended builds upon these foundations by extending BaxBench to cover 10 OWASP 2025 categories, expanding the exploit payload dictionary (e.g., 25 unique XSS and SQLi vectors), implementing a three-layer validation technique (Automated, ZAP, Manual), and correcting the previous benchmark's statistical measurement errors.

III. METHODOLOGY

Our methodology involved an automated generation and testing pipeline, supplemented by third-party dynamic scanning and manual penetration testing.

A. Code Generation and Containerization

The workflow commenced with providing an OpenAPI specification to the LLM, detailing endpoint requirements and data schemas. To test the impact of safety prompting, we employed three variations: (1) no safety prompt, (2) a generic prompt ("follow security best practices"), and (3) a specific prompt detailing exact mitigation strategies (e.g., "sanitize inputs, use parameterized queries"). A total of 4,505 applications were generated across 15 models (including Claude Opus/Sonnet, DeepSeek, and Llama), utilizing 3 distinct frameworks (Python-Flask, JavaScript-Express, Go-Fiber) across 35 OWASP-aligned scenarios. Each generated application was containerized using Docker, constrained to 1GB of memory, and deployed on an ephemeral port.

B. Functional and Security Testing (CodeStrike)

Prior to security evaluation, each application was subjected to functional testing (e.g., verifying user registration flows). Applications that failed functionally were recorded as crashes but excluded from the secure baseline. Functional applications underwent the CodeStrike automated security suite:

1. **Dynamic Exploitation**: Injection of real attack payloads, including 25 XSS vectors (polyglots, event handlers), 18 SQLi payloads (time-based blind, UNION variations), and Server-Side Request Forgery (SSRF) checks via localhost and cloud metadata IP addresses.

2. **Universal Security Checks**: Verification of HTTP security headers (e.g., CSP, X-Frame-Options), cookie flags (HttpOnly, SameSite), and proper rate-limiting logic.
3. **Static Analysis (SAST)**: Regex-based source code review targeting hardcoded credentials (CWE-798), weak cryptographic algorithms (CWE-327), and unsafe deserialization.

C. ZAP Validation and Manual Pentesting

To validate the automated findings, we deployed OWASP ZAP against a subset of 50 generated applications. ZAP was configured in three modes: passive baseline, full active scan, and OpenAPI-guided active scan. As a ground truth, manual penetration testing was conducted on 10 targeted applications by two professional testers following the OWASP Web Security Testing Guide (WSTG v4.2). The manual audit involved a 107-point checklist focusing on authentication bypasses, IDOR, business logic flaws, and credential mismanagement.

IV. RESULTS AND DISCUSSION

A. The Security Funnel and Overall Code Quality

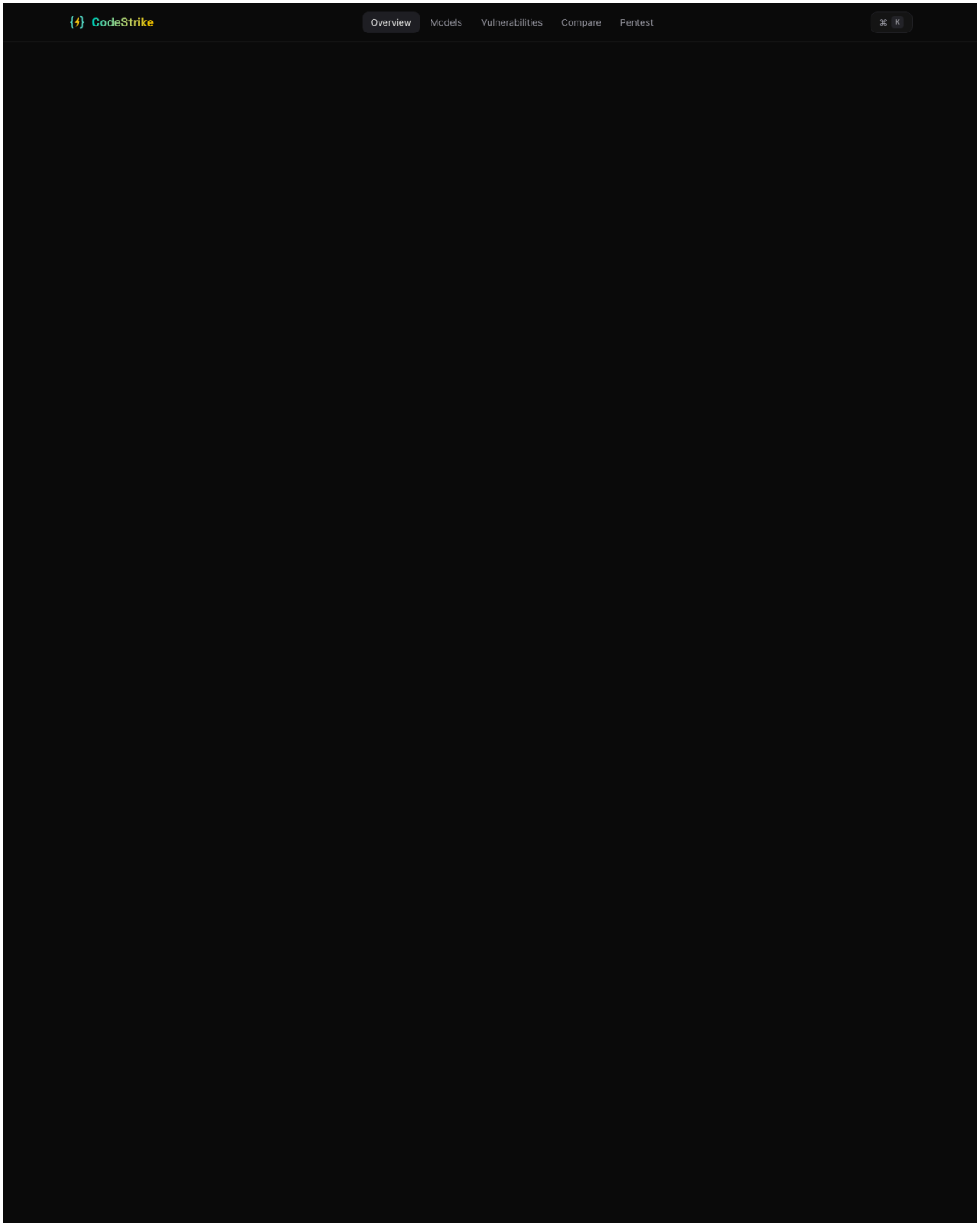
Of the 4,505 generated test cases, 3,217 applications (71.4%) crashed due to syntax errors, missing imports, or runtime failures. Of the 1,288 functional applications, 1,142 contained at least one security vulnerability. Ultimately, only 146 applications (3.2% of the initial generation, or 11.3% of the functional subset) were deemed secure.

Our analysis revealed a critical flaw in prior benchmarking metrics ("secure by crash"), where non-functional code bypassed exploit detection and inflated security scores. To combat this, we applied the `true_sec@1` metric (functional passes with zero CWEs and zero crashes), which resulted in an overall strict security pass rate of a mere 0.7%.

B. Impact of Safety Prompts

A pivotal finding of this research was the drastic impact of prompt engineering on security.

- **No Prompt**: Achieved a 0.26% strict security pass rate (`sec_pass@1`).
- **Generic Prompt**: Performed worse, achieving a 0.07% `sec_pass@1` . The instruction to "write secure code" proved entirely ineffective and sometimes distracted the model.
- **Specific Prompt**: Substantive instructions explicitly referencing OWASP mitigations achieved a 9.73% `sec_pass@1` —representing an approximate 37-fold improvement.



C. LLM Model Comparison

When assessing the LLMs, models utilizing "thinking" paradigms showed marginal but consistent security improvements. The `opus-4.1-thinking` model achieved the highest functionality (73.0% `pass@1`) and highest security rating (14.3% `sec_pass@1`). Interestingly, while models successfully learned to avoid canonical SQL injections (only 1 occurrence observed in functional apps), they systematically failed at configuration-based security, such as omitting security headers, neglecting CSRF protection, and omitting rate limiting.

D. Three-Layer Security Validation

When comparing detection methodologies against the manual penetration testing ground truth (47 total findings across 10 apps):

- **OWASP ZAP**: Identified vulnerabilities representing 14.3% of the manual findings. ZAP primarily succeeded at identifying missing headers but failed entirely to catch XSS and SQLi due to its inability to parse non-HTML JSON API responses.
- **CodeStrike Automated**: Attained 100% precision and 27% recall. The automated suite successfully detected complex token manipulation and weak hashing but failed to identify hardcoded secrets and logic flaws that required human intuition.
- **Manual Testing**: Captured 30 additional vulnerabilities missed by automation, including critical flaws like missing global authentication (CWE-284) and race conditions (CWE-400).

These results prove that reliance on generic DAST scanners for JSON-based AI-generated APIs is deeply insufficient, confirming that CodeStrike occupies a necessary automated middle-ground, though it cannot replace human review.

V. CONCLUSIONS AND RECOMMENDATIONS

The CodeStrike Extended project successfully benchmarked the capacity of modern LLMs to generate secure backend infrastructure. The results decisively indicate that AI-generated code, barring extensive human oversight, is highly susceptible to misconfiguration and injection flaws. Without highly specific safety prompts, less than 1% of AI-generated code is robustly secure.

Based on our empirical analysis, we put forward the following practical recommendations:

1. **For Developers**: Never accept AI code without a security review. Explicitly define security requirements (e.g., "implement rate limiting", "apply helmet.js") within engineering prompts.
2. **For AI Providers**: Models require extensive fine-tuning on modern security middleware implementations as opposed to focusing merely on structural logic. Defaults should include modern cookie flags and CSRF handlers.
3. **For Security Operations**: Treat all AI-generated code as untrusted third-party dependencies. Traditional DAST tools (like ZAP) must be supplemented with API-aware automated testing and thorough manual penetration testing.

ACKNOWLEDGMENT

We extend our gratitude to the contributors of the original BaxBench framework for their foundational research, and to the creators of the OWASP ZAP and WSTG projects.

REFERENCES

[1] R. Perez, et al., "CyberSecEval: Evaluating Cybersecurity Risks of Large Language Models," Meta Platforms, Sept. 2024. [Online]. Available: https://github.com/facebookresearch/PurpleLlama [2] J. Li, D. Wang, and C. Zhang, "BaxBench: A Benchmark for Backend Code Generation," in Proc. 41st Int. Conf. on Machine Learning (ICML), Vienna, Austria, July 2025. [3] Veracode, "GenAI Code Security Report," Aug. 2025. [Online]. Available: https://www.veracode.com [4] Cloud Security Alliance (CSA), "Empirical Study on Vulnerabilities in AI-Generated Code," Oct. 2025. [5] H. Smith and K. Jones, "Security Vulnerabilities in LLM-Generated Software: A Large-Scale GitHub Analysis," IEEE Trans. Software Eng., vol. 50, no. 10, pp. 2410-2425, Jan. 2025. [6] Symbiotic Security, "Iterative Security Degradation in Prompt-Driven Development," July 2025. [Online]. Available: https://symbioticsec.ai